

Ride-Hailing Demand & Smart Charging for an Electric Fleet

AAA Team Assignment, SS 2026

Team XX: Marius Meurer, Georgios ..., Anna Katharina Goitowski, Thanh Huyen Dao

2026-07-06

1 Problem Description

Business context and goal. A renowned German car manufacturer is establishing a ride-hailing platform operated with a fully electrified vehicle fleet, targeting the more receptive US market first. While the client masters vehicle manufacturing, it lacks the tactical and strategic know-how to operate an electric ride-hailing fleet. As the data science team of Bane & Wayne Partners, our goal is to give the client a data-driven understanding of ride-hailing demand for a representative US city in both spatial and temporal resolution, and to translate that understanding into fleet decisions on three levels. The strategic level concerns which districts to enter first. The tactical level concerns how many vehicles to deploy and at what times and locations. The operational level concerns how to charge and reposition the vehicles. Because the fleet is fully electric, operating cost also depends on how each vehicle is charged: charging at the right times keeps recharging cost low while still ensuring enough energy for the working day. This electrification angle connects the project to the course's broader focus on sustainability-oriented analytics, but in this assignment charging is treated narrowly, as a cost-minimisation question for the operator. Since no proprietary ride-hailing data exists yet, we use the City of Chicago taxi trip records (2024 onwards) as a proxy for ride-hailing demand. Taxis are a reasonable stand-in, although they differ from app-based hailing in booking behaviour and market coverage, a limitation we keep in view when drawing conclusions.

Data-mining goal. We translate this business goal into a sequence of analytics problems that together form the project. (1) *Data preparation*: turn the large, privacy-restricted raw trip records into clean, spatially and temporally aggregated demand panels, discretising the city with a hexagonal H3 grid and enriching it with external weather data. (2) *Descriptive spatial analytics*: characterise how demand and trip patterns vary across space and time at different resolutions, and locate demand hotspots using Gaussian Mixture Models and kernel density estimation. (3) *Predictive analytics*: frame demand as a supervised regression of the target `demand_count` per spatial unit and time bucket, conditioned on calendar, weather and location. This answers questions such as “what demand can we expect on a rainy Sunday from 10 to 11am at the outskirts of the city”, rather than forecasting the next hour from the current one, and it uses Support Vector Machines and feed-forward neural networks. (4) *Prescriptive analytics*: design a smart-charging agent via reinforcement learning that minimises recharging cost while avoiding the risk of running out of energy during the working day. (5) *Synthesis*: consolidate the findings into actionable recommendations for the client, including whether the operator should rely on private or public charging infrastructure. A guiding question throughout

is cost-effectiveness. We benchmark each model against simpler baselines and ask whether the added complexity, such as deep learning over a kernel SVM, is worth it for the client. Success is measured with standard regression metrics (MAE, RMSE, R^2) relative to a naive benchmark for the predictive models, and by recharging cost versus an energy-shortfall penalty for the charging agent.

2 Data description

Primary data: Chicago taxi trips. The project’s core dataset is the City of Chicago *Taxi Trips (2024–)* public record, used as a proxy for ride-hailing demand. It contains roughly 14.4 million individual trips spanning January 2024 to early April 2026 (extracted May 2026), with one row per trip and 23 raw columns. Each record provides an anonymised vehicle identifier (**Taxi ID**), start and end timestamps rounded to 15-minute intervals, trip duration in seconds (**Trip Seconds**) and distance in miles (**Trip Miles**), the operating company and payment type, and a fare breakdown (**Fare**, **Tips**, **Tolls**, **Extras**, **Trip Total**). Location is recorded at three nested spatial levels: Census Tract, Community Area (Chicago’s 77 administrative neighbourhoods), and pickup/dropoff centroid latitude-longitude. For privacy reasons the city suppresses the Census Tract for trips in sparsely travelled areas (missing for about 56% of pickups) and rounds timestamps. This directly shapes which fields are usable as spatial units (see Data preparation). For demand modelling, only the pickup-side time and location fields are required.

External weather data. Because weather is a strong short-term driver of mobility, the trip data is enriched with historical weather for Chicago from the Open-Meteo Historical Weather API (lat 41.85, lon -87.65), at hourly and daily resolution over the same 2024-2026 window. The variables include 2 m air temperature in F° , precipitation, rain and snowfall in inches, wind speed in mph, and the WMO weather code, plus daily sunshine duration and precipitation duration in hours. These are aligned to the trip timestamps so that weather-driven demand variation is not misattributed to the calendar.

Spatial reference. A GeoJSON of Chicago’s 77 community-area boundaries provides the geometry used to map and aggregate trips spatially and to anchor the H3 hexagonal grid introduced in the next section.

3 Data preparation

3.1 Data Cleaning

Data is technically cleaned before being analyzed to ensure that all columns have consistent formats and can be processed correctly. The main steps include: (1) standardizing column names; (2) converting timestamps into datetime format; (3) converting numeric and monetary columns into proper numeric types; and (4) checking for duplicate rows. This step ensures we have technically clean, processable data.

3.2 Missing Value Treatment

This section covers two steps: (1) investigating missing values and (2) defining the treatment strategy.

First, we examine the distribution of missing values to assess whether any patterns are present. In this dataset, missing values are not equally distributed across all spatial columns. Census Tract information has a very high share of missing values (55.65%). This is expected because the City of Chicago suppresses Census Tracts for some trips for privacy reasons. Therefore, Census Tracts are not suitable as the main spatial aggregation level. Community Area (2.79%) and centroid coordinates (2.74%) are much more complete and are therefore used as the main spatial units for the model. Crucially, the missing values for these variables are distributed uniformly across months and hours of the day, which means that excluding them does not distort the observed temporal demand pattern.

Dropoff variables have more missing values than pickup variables. Since the demand model focuses on where trips start, missing dropoff values are not used as grounds for removing observations from the main demand dataset.

Missing values are not removed globally, but are handled according to the specific validation or aggregation task. For the main modeling dataset — taxi demand defined by pickup location — we remove observations with null pickup coordinates. For side analyses such as trip duration, we exclude trips where duration equals zero.

3.3 Semantic Validation and Outlier Handling

After analyzing missing values, the next step is to verify whether the available values are meaningful from a business and domain perspective. We assess the reasonableness of trip start and end times, duration, distance, the relationship between distance and price, and trip fares by examining summary statistics (mean, median, and distribution) for each variable. We also confirm that pickup coordinates fall within the geographic boundaries of Chicago.

Based on this inspection, three hard filters are applied:

- **Impossible timestamps** (0.0007%): trips where the start timestamp is later than the end timestamp are excluded as physically impossible, most likely caused by data entry errors or timestamp rounding.
- **Excessive duration** (0.08%): trips with duration above 3 hours are removed, as this exceeds what is operationally plausible for an urban taxi ride in Chicago.
- **Extreme distance** (0.008%): trips with extreme distance values are excluded, as they fall outside any geographically plausible range within the Chicago metro area and are likely caused by GPS errors or data anomalies.

Together, these filters remove less than 0.1% of total trips, ensuring the modeling dataset reflects genuine demand without meaningfully reducing its size.

3.4 Aggregation

The cleaned trip-level data is aggregated along two dimensions: time and space.

(1) Temporal resolution — Time buckets are derived from the pickup timestamp. We evaluate four granularities: 1-hour, 4-hour, 8-hour, and daily intervals. Finer resolutions capture more operational detail but produce more volatile demand counts per cell; coarser resolutions are more stable but lose short-term signal. A 4-hour bucket strikes a practical balance — it is granular enough to distinguish morning, midday, evening, and night demand patterns, while remaining stable enough for reliable model training.

(2) Spatial resolution — We evaluate two spatial unit types: Community Areas and H3 hexagons. Community Areas are administratively defined and interpretable, but their irregular shapes and varying sizes limit spatial precision. H3 hexagons offer a uniform, standardized grid that can be tuned by resolution level (see Appendix). Resolutions range from R4 (~1,770 km², metro-wide) down to R8 (~0.74 km², a few city blocks). R6 (~36 km², roughly a large neighborhood) is selected as the primary spatial unit: it is fine enough to capture meaningful demand variation across Chicago’s districts, while finer resolutions such as R7 or R8 would produce many near-zero cells, making the prediction target sparse and noisy.

(3) Filling zero-demand combinations — A standard groupby only produces rows where at least one trip occurred, leaving (location, time) pairs with zero pickups absent from the dataset. These implicit zeros are made explicit via a Cartesian product of all cells and time buckets, so the model can learn from the absence of demand. Trip-characteristic columns are set to NaN for these filled cells.

Final decision: H3 R6 × 4-hour buckets is used as the primary resolution for all subsequent modeling. This combination provides neighborhood-level spatial granularity with a meaningful temporal structure, while keeping the dataset dense enough for robust model training.

4 Data analytics

4.1 Descriptive Analytics

- Brief recap of demand patterns across spatio-temporal resolutions (hex vs. census tract, hourly vs. 4-hourly)
- GMM/KDE hotspot results (1-2 key maps)

4.2 Predictive Analytics

4.2.1 Modeling dataset and feature design

This section constructs two supervised models that predict taxi trip demand per spatial unit and time bucket in spatio-temporal resolution. The canonical modeling dataset is the **H3 r6 x 4h aggregation**. The prediction target is `demand_count`, the number of trips originating in a spatial unit during a time bucket. The panel is zero-filled, so the absence of trips is represented as an observation of zero rather than a missing row; this is required for the model to learn low-demand conditions rather than only observed trips. Feature engineering process includes these steps:

Calendar encoding. Hour-of-day, day-of-week, and month are periodic quantities, so each is encoded as a sine/cosine pair rather than a raw integer. This preserves continuity across the period boundary (23:00 is adjacent to 00:00, December to January), which an ordinal encoding would misrepresent as maximally distant. A binary weekend indicator is added because demand shifts between commuting and leisure regimes that are not fully captured by the day-of-week cycle alone.

Spatial encoding. Each spatial unit is represented by its centroid latitude and longitude rather than by a one-hot indicator per cell. Continuous coordinates encode relative position and distance, allowing the support vector machine and neural network to share information between neighbouring units and to generalise across space — which directly supports location-conditioned queries such as demand “around the outskirts”. The centroid is derived deterministically from the H3 cell identifier, so reframing the retained spatial label as coordinates introduces no information loss.

Hotspot distances. The previous section identified three dominant demand centres, the downtown core, and O’Hare and Midway airports. The Euclidean distance from each unit to these three anchors is added as an explicit feature. These distances are deterministic transforms of the centroid coordinates and therefore add no new information in principle, but they supply a coordinate frame aligned with the observed demand structure, which finite-sample SVM and neural-network models exploit more readily than raw coordinates. Three additional columns is a negligible dimensional cost. The anchors are identified from training data only and are not hard-coded. Distances are expressed in miles for consistency with the trip-level distance fields.

Weather data. The weather features are carried over exactly as aggregated in Section 1 with any redundancy (precipitation overlapping rain and snow) and the encoding of the categorical weather code left to the model stage.

Leakage control. The per-bucket trip aggregates (`avg_trip_miles`, `avg_trip_seconds`, `avg_trip_total`, `n_unique_taxis`) are excluded from the feature matrix: they are computed from the same trips that define the target and are therefore unavailable at prediction time. Consistent with the task framing, no lagged or same-bucket demand of the target enters the features; prediction is conditioned only on calendar, weather, and spatial position. The constant metadata columns (`spatial_resolution`, `temporal_resolution`) are also dropped.

4.2.2 Validation strategy

Because the target is demand conditioned on calendar, weather, and location — not on past demand — this is a supervised regression problem rather than a forecasting one. The validation design therefore uses cross-validation for full seasonal coverage, with one adjustment for the non-independence of observations.

Grouping by day. The 4-hour buckets of a given day and cell share weather, calendar values, and unobserved local shocks. A purely random split could place some of a day’s buckets in training and the rest in test, letting the model exploit day-specific signal absent from the features and inflating the score. All splits are therefore **grouped by calendar day**: a day appears entirely in one partition or the other, never both.

Final holdout. A grouped random sample of whole days (15%) is set aside as an untouched test set, used only for the SVM-vs-NN comparison. Drawing days across the full sample keeps every season represented in the holdout, which a single trailing time-block would not.

Cross-validation. Model selection uses `GroupKFold` (5 folds, group = day) over the development set. the holdout is never touched during tuning.

Fit-on-train-only. The hotspot anchors and feature standardisation are fitted on training data only. Standardisation and the target transform are wrapped in a pipeline so they are refit inside each CV fold automatically.

Target transform. Demand is heavily right-skewed with 12.6% zeros, so it is modelled on a `log1p` scale and predictions are inverted before scoring.

Metrics. MAE, RMSE, and R^2

4.2.3 Support vector machines

- + Model progression. (For reference: linear $\rightarrow$ kernel (RBF), and how you selected hyperparameter)
- + Result of the performance evaluation metrics (e.g., RMSE, MAE, R^2)
- + Comment on its shortfall

4.2.4 Neural networks

We also use feedforward neural network method to predic taxi demand. Our model development process includes developing base line model and tuning hyperparameter.

First, feedforward baseline model is the model without tuning hypterparameter. It uses the same feature matrix, splits and scaling as SVM model. Regarding architecture, the input layer sized to the feature count, two hiddens layers with correspondingly 64 and 32 nodes with ReLU as activation function (the detail of architecture can be found in the Appendix). Other hyperparameters in this model include:

Dropout = 0.2 - it means that in each running, model omits 20% nodes in the hidden layers. It can reduce the interaction between some high-correlated nodes and make sure model does not extremely depend on some specific parameter.

Optimiser. Adam with `lr=1e-3`. By using Adam for optimiser, each parameter gets its own adaptive step size, scaled down where gradients are large/noisy, scaled up where they're small/consistent.

Early stopping. Patience=15 epochs on an internal 10% validation split drawn from the training fold. This prevents overfitting without requiring a fixed epoch budget, and the best weights are restored before the model is returned. The internal `val_faction` split used for early stopping. It means that we use a part of training dataset to train, and the internal validation part to tell the model when to stop. This internal validation fold does not overlap with the outer test fold or the final holdout. The result of baseline model is shown in Appendix.

Second, we try model with different number and width of layers, dropout, learning rate, batch size, and epochs. Hyperparameters were tuned via manual grid search over the following candidate values is shown in the Appendix.

Among the candidate configurations evaluated via grid search, the best-performing model was found with hidden sizes (128, 64, 32), dropout of 0.1, learning rate of 1e-3, and batch size of 512. The average `demand_count` of the dataset is 108. The MAE of the best model is 22, which is only 20% of average. It indicates that model is quite good in predicting the demand. Meanwhile, RMSE is much

more larger than MAE. It may be because the model makes huge mistakes with the spatial unit that have very high demand. $R^2 = 93.54$. It means that model can explain 93.54% of variance. In other word, it shows that model is 93.54% than the model predicts all rows as mean of demand.

To see how the variables contribute to the prediction, we analyze SHAP value (Shapley Value Foundation), which indicates how much this feature pushed the prediction up (+) or down (-). The SHAP value of feature i is calculated by: at every possible subset of features, add feature i to that subset, and measure how much the prediction changes. Then average that change across all subsets, weighted by how many ways that subset could occur.

The SHAP plot can be found in the Appendix. The most important features are related to spatial unit, especially the **distance** from the hexagon to the busy areas. Low distance to Loop (blue dot) makes the demand increase, which means the hexagon near the busy zones have high demand. It aligns with the business sense. The **time features** (`hour_cos`, `hour_sin`) also have the high ranks, which means time of day matters a lot. `is_weekend`: red dot (`weekend = 1`) slightly negative. It means that weekdays have slightly higher taxi demand overall. The **weather features** do not have much contribution to the demand, the SHAP value is nearly 0.

4.2.5 Resolution sensitivity

+ How model's performance varies as we decrease or increase temporal or spatial resolution or w

4.2.6 SVM vs. NN comparison

- + A side-by-side holdout table (SVM vs NN) with MAE, RMSE, R^2 and cost
- + An explicit verdict on whether a deep-learning approach is warranted here
- + Improvement levers for a follow-up project
- + A non-technical translation of the chosen model's error into a fleet-operations implication

Appendix: - Model result: Report a table across resolutions (hex vs. census tract, different temporal bins)

5 Appendix

5.1 H3 hexagon area corresponding to specific resolution

Resolution	Avg Area	Rough Scale
R4	~1,770 km ²	Large city / metro region
R5	~253 km ²	City district
R6	~36 km ²	Large neighborhood
R7	~5.2 km ²	Neighborhood
R8	~0.74 km ²	A few city blocks

Need to add: percentage of zero cells for each combination of temporal, spatial unit

5.2 Baseline Neural Network Architecture

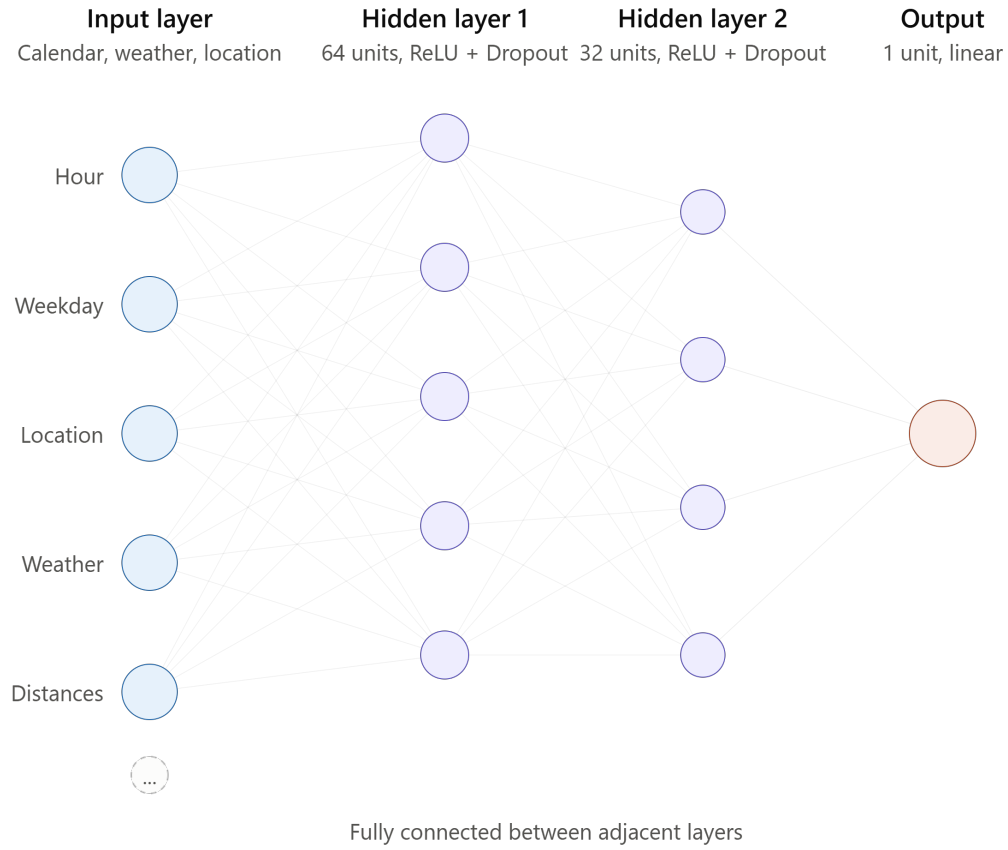


Figure 1: Model architecture

5.3 candidate values of hyperparameters for tuning Neural Network model

Parameter	Candidate Values
<code>hidden_sizes</code>	(64, 32), (128, 64, 32)
<code>dropout</code>	0.1, 0.2
<code>lr</code>	1e-3, 5e-4
<code>batch_size</code>	256, 512

5.4 Baseline Neural Network model result

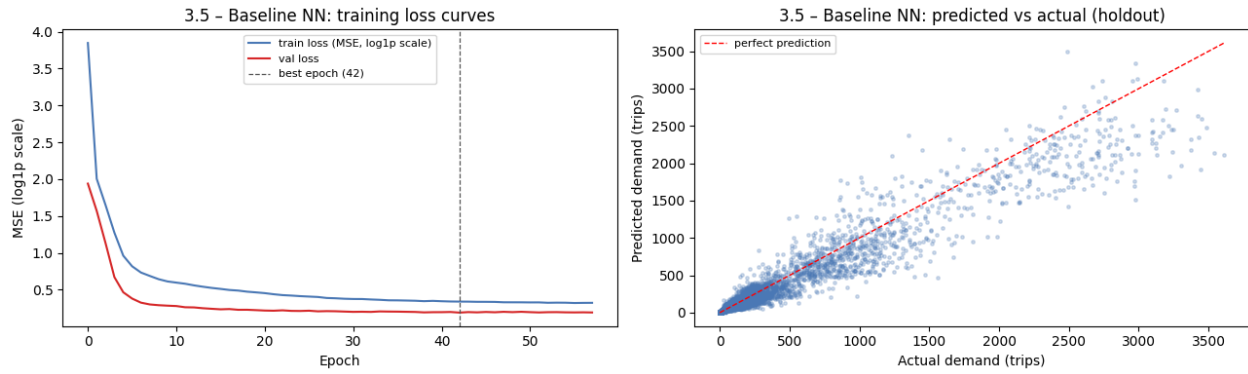


Figure 2: Model result

5.5 Best Neural Network model

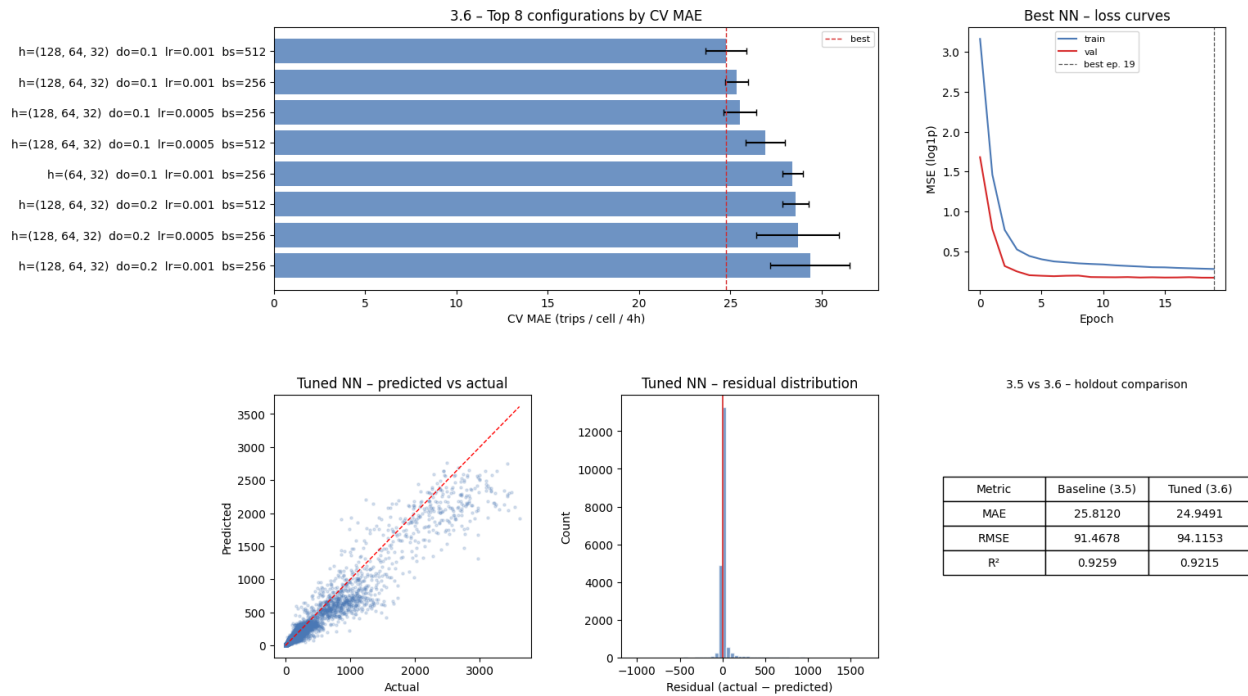


Figure 3: Model result

5.6 Feature importance - SHAP - of best Neural Network model

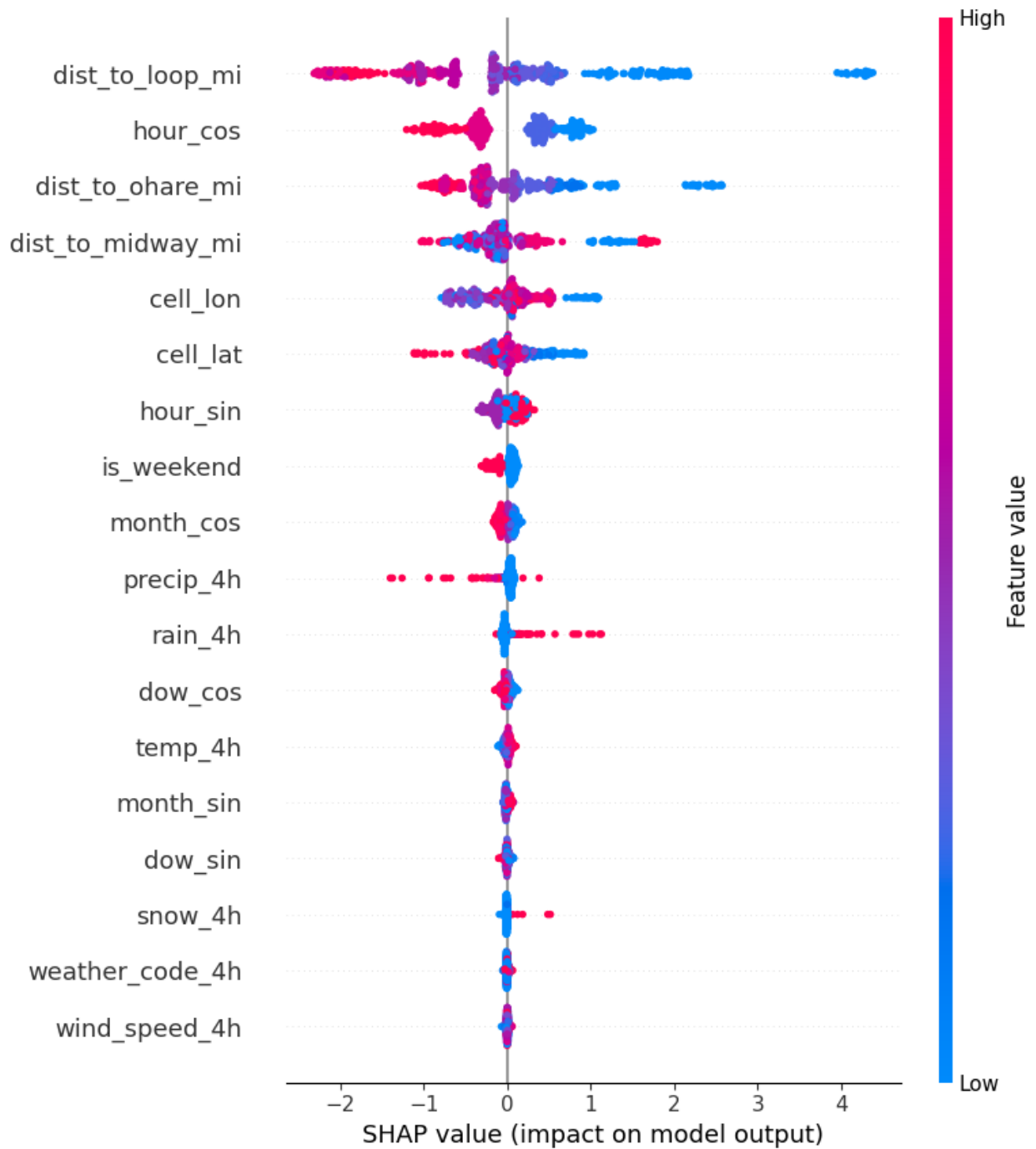


Figure 4: SHAP